APUNTESHUB · DOCUMENTO TÉCNICO

Mejoras de Inteligencia Artificial

Cómo evolucionó el motor de IA de la plataforma: búsqueda semántica, búsqueda tolerante a errores, estudio interactivo con IA y respuestas fundamentadas exclusivamente en cada apunte.

Embeddings multilingües | Búsqueda híbrida | Meilisearch | Estudio con IA | BYOK | Grounding & seguridad

---

Período cubierto: **26 de mayo → 15 de junio de 2026**
Audiencia: docentes y evaluadores · Nivel: semi-técnico

0 Resumen ejecutivo

En tres semanas el sistema de IA pasó de una búsqueda puramente semántica a un motor de varias capas que entiende lo que el alumno quiere decir aunque lo escriba mal, encuentra el contenido correcto combinando significado y palabras exactas, y genera material de estudio interactivo sin inventar contenido.

BÚSQUEDA

Híbrida

Significado (vectores) + texto exacto (Postgres FTS), con tolerancia a acentos y errores de tipeo.

ESTUDIO

V/F · MC

Verdadero/Falso y multiple choice generados desde el apunte, con validación estricta del formato.

MODELOS

BYOK

Cada usuario puede usar su propia API key (OpenAI / Gemini), cifrada con AES-256-GCM.

CONFIABILIDAD

Grounding

El chat solo responde con lo que está en el apunte; los ejemplos extra se marcan como ilustrativos.

Contenido

1. 1 Embeddings: cómo representamos el significado del texto
2. 2 Búsqueda híbrida: significado + palabras exactas
3. 3 Tolerancia a errores: "aunque escribas mal, te lo encuentra"
4. 4 Meilisearch: búsqueda pública en Explorar
5. 5 Estudiar con IA: preguntas, V/F y multiple choice
6. 6 BYOK: usar tu propia API key de forma segura
7. 7 Restricciones: qué se puede preguntar y qué no
8. 8 Respuestas sin invención: grounding del chat

1 Embeddings MEJORADO

Un **embedding** es la representación numérica del significado de un texto: una lista de 1024 números (un "vector"). Dos textos que hablan de lo mismo quedan cerca en ese espacio aunque usen palabras distintas. Es la base de toda la búsqueda semántica.

Qué usamos

Generamos los embeddings con el modelo `embed-multilingual-v3.0` de Cohere, que produce vectores de **1024 dimensiones** y entiende español nativamente. Cada fragmento de apunte y cada pregunta del usuario se convierten al mismo espacio vectorial, de modo que se pueden comparar entre sí.

Parámetro	Valor	Por qué
Modelo	`embed-multilingual-v3.0`	Calidad en español, sin depender de traducción.
Dimensiones	1024	Buen equilibrio entre precisión y costo de almacenamiento.
Ventana del modelo	512 tokens	Límite máximo que el modelo lee de un fragmento.
Tipo de entrada	`search_document` / `search_query`	Cohere optimiza distinto un documento que una consulta.

Las mejoras que hicimos

a) Fragmentación ("chunking") calibrada al modelo

Antes de vectorizar, cada PDF se divide en fragmentos. El problema: si un fragmento supera los 512 tokens del modelo, el resto se descarta silenciosamente y se pierde información. Calibramos el partidor para apuntar a **~400 tokens (~1600 caracteres)** por fragmento, con un tope duro de ~2000, de modo que cada fragmento entre completo en el modelo.

- Corta respetando la estructura: primero por **secciones**, después por párrafos, después por oraciones y solo en último caso por caracteres.
- Mantiene un **solapamiento (~300 caracteres)** entre fragmentos consecutivos para no cortar una idea por la mitad.
- Guarda metadata por fragmento: **página de inicio y fin, y título de sección**. Esto es lo que después permite citar "página 4-5" en el chat.

b) Normalización de acentos antes de vectorizar

"Le Châtelier" y "Le Chatelier" deberían producir el mismo significado. Antes de enviar el texto al modelo le quitamos los diacríticos (tildes y diéresis), así ambas formas generan vectores compatibles. **El contenido original se guarda intacto** en la base de datos: la normalización es solo para la comparación.

c) Procesamiento por lotes, en paralelo y resistente a fallos

Vectorizar un apunte largo son cientos de fragmentos. Para que sea rápido y robusto:

- **Lotes** de hasta 96 fragmentos por llamada al proveedor.
- **Concurrencia controlada** (varios lotes a la vez, configurable) sin saturar el límite de la cuenta.
- **Reintentos con backoff exponencial** (1s → 2s → 4s + variación aleatoria) ante errores temporales (límite de tasa, timeout, caída momentánea). Los errores definitivos —clave inválida, cuota agotada— no se reintentan.
- Reensamblado en orden: aunque los lotes terminen desordenados, cada vector vuelve a su fragmento correcto.

En una línea para un profesor

Convertimos cada apunte en "coordenadas de significado" usando un modelo que entiende español, lo hacemos en pedazos del tamaño justo para no perder texto, y de forma robusta para que un apunte de 200 páginas se procese sin fallar.

2 Búsqueda híbrida NUEVO

La búsqueda semántica sola tiene un punto débil: a veces el alumno busca una palabra *exacta* (una fórmula, un nombre propio, una sigla) que el significado general no prioriza bien. La solución es combinar dos motores.

Los dos motores

Motor	Qué hace	Fortaleza
Vectorial (pgvector)	Compara el significado de la consulta con el de cada fragmento usando los embeddings.	Encuentra contenido relacionado aunque use otras palabras ("calor" ↔ "energía térmica").
Texto completo (Postgres FTS)	Busca las palabras de la consulta tal cual aparecen en el texto, con raíces del español.	Aciertos exactos: nombres propios, términos técnicos, siglas.

Cómo los combinamos

Implementamos la búsqueda híbrida dentro de la base de datos (PostgreSQL), en una función `search_chunks_hybrid_for_study`. Cada fragmento recibe un **puntaje combinado** con pesos:

> **Puntaje final** = 0.65 × (puntaje semántico) + 0.35 × (puntaje de texto)
>
> *En el estudio el peso semántico domina (0.65) porque entender la intención importa más; el componente textual (0.35) rescata las coincidencias exactas que el vector podría dejar pasar.*

El recorrido de una consulta del chat de estudio es:

1. **Reescritura de la pregunta** → si es vaga ("dame un ejemplo de eso") se completa con el tema anterior de la charla.
2. **Embedding de la consulta** → se convierte la pregunta al espacio vectorial.
3. **Búsqueda híbrida** → vector + texto, solo sobre apuntes que el usuario compró (filtro de permisos en la propia consulta).
4. **Expansión por vecinos** → se suman los fragmentos anterior y siguiente para no perder contexto.
5. **Recorte de contexto** → se limita a 18 fragmentos / ~12.000 caracteres y se arma la respuesta.

Diseñada para no fallar nunca del todo

- Si la búsqueda híbrida falla por cualquier motivo, el sistema **cae automáticamente** a la búsqueda vectorial anterior ("M1"). El usuario siempre obtiene resultados.
- Doble umbral de similitud: arranca exigente (0.30); si trae pocos resultados, baja el umbral (0.24) y vuelve a buscar para no dejar al alumno sin respuesta.
- Los permisos viven dentro de la consulta SQL: solo se devuelven fragmentos de apuntes con compra `completed`. La búsqueda no puede filtrar contenido que no pagaste.

> **Beneficio**
>
> Más precisión que la búsqueda semántica sola, sin perder la capacidad de entender sinónimos; y a la vez robustez: si algo falla, degrada con elegancia en vez de romperse.

3 "Aunque escribas mal, te lo encuentra"

Este comportamiento que el usuario percibe como "magia" es en realidad la suma de varias capas trabajando juntas. No hay una sola pieza; hay defensa en profundidad contra los errores de escritura.

Capa	Mecanismo	Qué error corrige
Significado	Embeddings semánticos	Palabras distintas, mismo concepto. La escritura exacta no importa.
Acentos	`unaccent` en la indexación y normalización antes de vectorizar	"le chatelier" encuentra "Le Châtelier".
Raíces (stemming)	Diccionario español de Postgres FTS	"equilibrios" / "equilibrar" encuentran "equilibrio".
Errores de tipeo	Tolerancia a typos de Meilisearch (búsqueda pública)	"le chatlier", "termodinmica" siguen acertando.
Nombres propios	Aliases académicos curados por reglas	Encuentra "Le Chatelier" aunque el PDF no lo nombre literalmente.
Texto OCR	Instrucción explícita al modelo de chat	Interpreta apuntes manuscritos digitalizados con errores de reconocimiento.

Aliases académicos: el caso "Le Chatelier"

A veces el concepto central de un apunte tiene un nombre propio que el PDF no escribe literalmente (o la digitalización lo rompió). Para que igual se encuentre, derivamos **aliases** a partir de señales del contenido —no del contenido en sí— con reglas conservadoras:

- Exige una **señal ancla** obligatoria (el tema central, p. ej. "equilibrio").
- Exige al menos una **señal específica** del dominio (química, "desplazamiento del equilibrio") y un mínimo de señales totales.
- Esto evita falsos positivos: un apunte de Física que menciona "equilibrio", "temperatura" y "presión" *no* dispara el alias de química, porque le falta la señal específica.

Resultado: si el apunte realmente trata de Le Chatelier, agregamos términos como "principio de le chatelier" o "desplazamiento del equilibrio" al índice de búsqueda, sin contaminar apuntes de temas vecinos.

> **Para explicar a un profesor**
> El sistema no busca "la palabra que tipeó el alumno": busca el *significado*, ignora acentos, reconoce raíces de palabras, tolera letras cambiadas y conoce los nombres propios del tema. Por eso un error de tipeo no rompe la búsqueda.

4 Meilisearch en Explorar NUEVO

Meilisearch es un motor de búsqueda especializado, muy rápido y con excelente tolerancia a errores de tipeo. Lo incorporamos para mejorar la **búsqueda pública** de apuntes en la sección Explorar (browse), detrás de un interruptor de activación (feature flag).

Qué problema resuelve

La búsqueda pública necesita ser instantánea y perdonar errores de escritura del visitante que todavía no compró nada. Meilisearch está diseñado exactamente para eso: responde en milisegundos y corrige typos de fábrica.

Cómo lo implementamos

- **Índice de apuntes:** por cada apunte publicado indexamos título, materia, descripción, carrera, universidad, nombre de archivo y, sobre todo, los **temas declarados por el vendedor**, los **títulos de sección del propio apunte** (su índice/tabla de contenidos, campo `sectionTitles`), términos derivados del contenido (`chunkTerms`) y aliases académicos.
- **Prioridad de campos:** el título y los temas pesan más que las señales derivadas; los títulos de sección van justo debajo de los temas (son los encabezados reales del material), por encima de `chunkTerms`. Así lo que estructura el autor manda en el ranking.
- **Tolerancia a typos calibrada:** activada y un poco más permisiva que el default, para tolerar errores como "le chatlier".
- **Interruptor de seguridad:** solo se usa si el feature flag está activo *y* está configurado; si no, la búsqueda cae automáticamente a PostgreSQL.
- **Timeout corto (5s):** si Meilisearch está lento o caído, la búsqueda degrada a Postgres en vez de quedarse colgada.

El índice del apunte ahora alimenta la búsqueda NUEVO

Detectamos que un apunte recién ingestado a veces no aparecía al buscar por un tema que sí estaba en su contenido. El motivo: la búsqueda pública se apoyaba sobre todo en los **temas que cargaba el vendedor a mano**, mientras que el índice/tabla de contenidos del PDF —que el procesador de IA ya extrae como **títulos de sección** de cada fragmento— no se usaba como señal. Lo conectamos en dos frentes complementarios:

- **Como campo de búsqueda (`sectionTitles`):** los títulos de sección se indexan directamente en Meilisearch, limpios y deduplicados (se descartan encabezados genéricos como "Unidad 1" o "Introducción"). No es contenido del PDF: son solo los encabezados, equivalentes a los temas del índice.
- **Como temas automáticos:** al ingestar, esos títulos se agregan a los "Temas del apunte" *sin pisar* los que cargó el vendedor (solo rellenan los lugares libres). Esto ayuda también a la búsqueda de respaldo en PostgreSQL, donde los temas pesan en el ranking.

> **Para apuntes ya cargados, sin reprocesar**
> Los apuntes nuevos se completan solos al ingestar. Para los que ya estaban subidos agregamos un **"backfill"**: lee los títulos de sección que ya están guardados en la base y los vuelca a los temas *sin volver a bajar el PDF ni re-generar embeddings* (lo costoso), y reindexa al instante. Una sola operación de administración pone al día todo el catálogo.

Decisiones de seguridad y privacidad

> **El índice NO es la fuente de la verdad**
> Meilisearch devuelve **solo los IDs** de los apuntes que coinciden, ordenados por relevancia. Esos IDs **siempre se revalidan contra la base de datos** (¿sigue publicado?, ¿no fue borrado?) antes de mostrarse. El índice acelera, pero los permisos los decide siempre Supabase.

- **No indexamos el contenido completo** de los apuntes: solo señales de búsqueda acotadas (términos cortos, máximo 50 por apunte). Nunca párrafos enteros.
- El módulo es **solo de servidor**: la API key de Meilisearch jamás llega al navegador.
- Los logs nunca incluyen contenido, fragmentos, embeddings ni claves.

Beneficio	Impacto
Velocidad	Respuestas de búsqueda en milisegundos en Explorar.
Tolerancia a typos	El visitante encuentra apuntes aunque escriba mal.
Recall por temas	Encuentra apuntes por los temas declarados *y por los títulos de sección del propio índice del apunte*, aunque el título general no los nombre.
Riesgo acotado	Activable/desactivable sin tocar código; degrada a Postgres ante fallos.

¿Por qué NO usamos Meilisearch en el chat de IA?

Es una decisión deliberada de diseño, no una omisión. La búsqueda pública (Explorar) y la recuperación del chat son **dos problemas distintos**, y cada motor está en el que le corresponde. Meilisearch potencia el browse; el chat se apoya en la búsqueda híbrida de Postgres (pgvector + FTS).

Motivo	Explicación
Significado vs. palabras	El chat necesita traer fragmentos *conceptualmente* relacionados con la pregunta aunque no compartan palabras ("calor" ↔ "energía térmica"). Eso lo dan los embeddings. Meilisearch es, en esencia, un motor léxico (matchea palabras con tolerancia a typos): es excelente para el browse, pero no es la capacidad central que el chat requiere.
Seguridad y privacidad	El chat busca solo en chunks de apuntes que el usuario **compró**, y ese filtro vive dentro de la consulta SQL: el contenido nunca sale de Postgres. Para alimentar al modelo, el chat necesita el **texto completo** del fragmento. Llevarlo a Meilisearch obligaría a duplicar todo el contenido pago en un índice externo y a reimplementar ahí la autorización por usuario, justo lo que el browse evita (en Explorar Meili no indexa contenido, solo señales cortas, y devuelve solo IDs que se revalidan contra Supabase).
Una sola fuente de verdad	Los chunks y sus embeddings ya viven en Postgres con índice vectorial. Recuperar ahí evita una segunda copia para sincronizar, ventanas de inconsistencia y un punto de falla extra, sobre contenido pago donde la correctitud importa.
La velocidad no es el cuello de botella	La fortaleza de Meilisearch es responder en milisegundos. Pero el chat está dominado por la llamada al modelo de lenguaje (cientos de ms a segundos): la latencia de recuperar en Postgres es despreciable al lado de eso.

> **¿Y si hiciera falta más tolerancia a typos en el chat?**
> El único punto donde Meilisearch agregaría algo es tolerar términos exactos mal tipeados (una fórmula, un nombre propio). Pero el embedding de la pregunta ya es robusto a errores y el FTS aporta `unaccent` + raíces del español. Si alguna vez se midiera como un problema real, la solución barata es activar `pg_trgm` (similitud por trigramas) **dentro de Postgres**, sin sacar contenido de la base ni sumar infraestructura.

5 Estudiar con IA: preguntas, V/F y multiple choice NUEVO

Separamos el "chat de preguntas" del "modo estudio". El modo estudio genera **actividades interactivas** a partir del apunte: verdadero/falso, multiple choice, mini-quiz, explicaciones paso a paso, etc.

Los modos de respuesta

Modo	Qué genera
`true_false`	Afirmaciones para marcar V o F, mezclando verdaderas y falsas, con explicación.
`multiple_choice`	Preguntas con 4 opciones, una sola correcta y distractores plausibles.
`quiz`	Mini-quiz de 3 a 5 preguntas abiertas para practicar.
`explain` / `step_by_step`	Explicación didáctica, opcionalmente descompuesta en pasos.
`summary` / `beginner`	Resumen breve o lenguaje para principiantes.

El desafío técnico: salida estructurada confiable

Para que el frontend dibuje tarjetas interactivas necesita un **JSON con un formato exacto** (lista de preguntas, opciones, índice de la correcta, explicación). Pero los modelos de lenguaje a veces "decoran" la respuesta con texto extra, bloques de código o markdown. Resolvimos esto con tres capas:

1. **Prompt dedicado y estricto** → para V/F y MC usamos un prompt minimalista que pide JSON puro, sin web ni texto adicional, con el esquema exacto.
2. **Parseo tolerante** → si el modelo igual agrega texto, extraemos el primer objeto JSON balanceado (respeta llaves dentro de strings) o lo que venga entre ```` ```json ````.
3. **Reintento de reparación** → si no se pudo parsear, se le pide al modelo una vez más, recordándole que devuelva solo JSON.

Validación: nunca confiamos a ciegas en el modelo

Antes de mostrar una actividad, validamos cada pregunta:

- Multiple choice: **exactamente 4 opciones** y un índice de correcta válido (0 a 3). Si no, se descarta esa pregunta.
- Verdadero/Falso: el campo `answer` debe ser un booleano real, con afirmación y explicación no vacías.
- Tope de preguntas (3 o 5 según lo pedido), longitudes máximas y limpieza de cada campo.
- **Fallback elegante:** si no se pudo armar ninguna actividad válida, en vez de mostrar JSON roto se ofrece un set de preguntas genéricas para practicar.

> **Por qué importa**
>
> El alumno siempre ve una actividad bien formada o un mensaje útil: nunca un error técnico ni JSON crudo. Y las preguntas se basan solo en el contenido de su apunte.

6 BYOK: tu propia API key, de forma segura NUEVO SEGURIDAD

BYOK ("Bring Your Own Key") permite que cada usuario conecte su propia clave de OpenAI o Gemini. El chat usa entonces el modelo del usuario en lugar del modelo de la plataforma. El reto central es guardar esa clave de forma segura.

Cifrado de las claves

Las API keys son secretos sensibles. Nunca se guardan en texto plano. Usamos **AES-256-GCM**, un cifrado autenticado estándar de la industria:

- Cada clave se cifra con un **vector de inicialización (IV) aleatorio** distinto, generado por clave.
- Se guarda también una **etiqueta de autenticación** que detecta si el dato fue alterado: si alguien toca el cifrado, el descifrado falla en vez de devolver basura.
- En la base de datos solo queda el texto cifrado, el IV, y una **pista** de los últimos 4 caracteres (p. ej. `...a8f2`) para que el usuario reconozca su clave sin exponerla.

Flujo de alta de una clave

1. El usuario pega su clave y elige proveedor (OpenAI / Gemini / custom).
2. **Validamos contra el proveedor** → hacemos una llamada de prueba; si la clave no funciona, se rechaza antes de guardar nada.
3. **Ciframos** (AES-256-GCM) y guardamos solo el cifrado + IV + pista.
4. Límite de **5 claves por usuario**; cada fila protegida por Row Level Security (solo el dueño la ve).

Plataforma vs. modelo propio

	Modelo de la plataforma	Modelo propio (BYOK)
Proveedor	Groq (`llama-3.3-70b`)	OpenAI o Gemini del usuario
Costo	Lo paga la plataforma	Lo paga el usuario en su cuenta
Búsqueda web	No	Sí (salvo en V/F y MC)
Mismas reglas de grounding	Sí	Sí

Garantía de seguridad

La clave se descifra **solo en el servidor y solo en el momento** de llamar al modelo del usuario. Nunca viaja al navegador, nunca aparece en logs, y se valida su pertenencia (debe ser del usuario autenticado y estar activa) antes de usarse.

7 Qué se puede preguntar (y qué no) SEGURIDAD

El chat es una herramienta de estudio sobre apuntes que el usuario compró, no un canal para extraer el contenido completo ni para manipular al modelo. Pusimos controles en la entrada, antes de gastar un solo recurso de IA.

Filtro anti-extracción y anti-manipulación

Antes de procesar cualquier pregunta, un guardia detecta intentos de:

- **Volcado de contenido:** "transcribí todo", "dame todo el contenido", "listá todo", "pasame todo". Se rechazan: el chat responde preguntas puntuales, no copia el apunte completo.
- **Inyección de instrucciones (prompt injection):** "ignorá tus instrucciones", "olvidá las reglas", "actuá como…", "system:". Se bloquean para que nadie reprograme al asistente.

Límites de entrada

Control	Valor	Motivo
Largo máximo de pregunta	500 caracteres	Evita abusos y prompts gigantes.
Apuntes por consulta	10 máx.	Acota el contexto y el costo.
Historial considerado	Últimos 6 mensajes	Mantiene contexto sin inflar el prompt.
Límite de tasa	20 pedidos / minuto por usuario	Protege el servicio de un uso abusivo.

> **Defensa en dos frentes**
> Estos filtros viven en la *entrada* (se rechaza antes de llamar a la IA) y se refuerzan en el *prompt del sistema* (reglas que el modelo no puede saltear). Aunque una capa falle, la otra sostiene.

8 Respuestas sin invención: el chat se basa solo en el apunte

El objetivo: que el asistente responda **solo con lo que dice el apunte** (sin inventar datos), pero que igual sea un buen tutor —que explique con claridad y dé ejemplos ilustrativos cuando ayudan—. Esto se llama *grounding* (respuesta anclada a una fuente).

Las reglas que le imponemos al modelo

El prompt del sistema fija reglas estrictas que ninguna instrucción del usuario puede modificar:

- **Solo el contexto:** responde únicamente con información presente en los fragmentos recuperados del apunte. Si la pregunta no se relaciona, dice que no encontró esa información.
- **No copia textual:** nunca reproduce más de una oración literal; siempre parafrasea y sintetiza. Esto protege el contenido del vendedor.
- **Ejemplos marcados:** puede dar ejemplos didácticos propios, pero debe rotularlos como **"Ejemplo ilustrativo:"** para distinguir lo que está en el apunte de lo que agrega el tutor.
- **Citas honestas:** solo cita "página X / sección Y" para afirmaciones realmente apoyadas por ese fragmento. **Nunca inventa números de página.**
- **Tolerante a OCR:** se le avisa que los apuntes pueden venir de manuscritos digitalizados, con errores de reconocimiento, para que interprete con flexibilidad.

Estructura de respuesta de tutor

Cuando es natural, la respuesta se separa en secciones claras:

Sección	Contenido
Según el apunte	Solo afirmaciones apoyadas por los fragmentos.
Esto significa	Interpretación o consecuencias derivadas del contexto.
Ejemplo ilustrativo	Ejemplo del tutor, explícitamente marcado como agregado.
Para practicar	Ejercicios o preguntas basadas en el contenido.

Refuerzos a nivel sistema

- **Las páginas se adjuntan de forma determinística:** no confiamos en que el modelo informe bien las páginas; las calculamos nosotros a partir de la metadata de los fragmentos usados y las mostramos como fuente.
- **Conexión con temas declarados:** si el alumno usa un término que no está literal en el apunte pero coincide con un tema declarado, el modelo aclara "no aparece escrito así, pero el concepto corresponde a…" y lo conecta solo con lo que hay en el contexto.
- **Temperatura baja (0.3):** menos creatividad libre, más fidelidad a la fuente.

> **El equilibrio que logramos**
>
> Fiel a la fuente *y* didáctico: responde con lo que dice el apunte, cita páginas reales, no copia el contenido textualmente, y cuando aporta un ejemplo propio lo deja claramente identificado. No inventa, pero tampoco se queda en repetir.